

Semantic-Aware Transmission Scheduling: A Monotonicity-Driven Deep Reinforcement Learning Approach

Jiazheng Chen¹, Graduate Student Member, IEEE, Wanchun Liu¹, Member, IEEE,
Daniel E. Quevedo², Fellow, IEEE, Yonghui Li¹, Fellow, IEEE, and Branka Vucetic¹, Life Fellow, IEEE

Abstract—For cyber-physical systems in the 6G era, semantic communications connecting distributed devices for dynamic control and remote state estimation are required to guarantee application-level performance, not merely focus on communication-centric performance. Semantics here is a measure of the usefulness of information transmissions. Semantic-aware transmission scheduling of a large system often involves a large decision-making space, and the optimal policy cannot be obtained by existing algorithms effectively. In this letter, we first establish the monotonicity of the Q function of the optimal semantic-aware scheduling policy and then develop advanced deep reinforcement learning (DRL) algorithms by leveraging the theoretical guideline. Our numerical results show a 30% performance improvement compared to benchmark algorithms.

Index Terms—Deep reinforcement learning, sensor scheduling, age of information, semantic communications.

I. INTRODUCTION

DIFFERENT from the first generation to the 5G communications, which focused on improving data-oriented performance (e.g., data rate, latency and reliability), the 6G is envisioned to support many emerging applications and adopt semantic-related metrics for application-level performance optimization [1]. The potential applications supported by 6G semantic communications can be divided into two categories: human-centered systems (e.g., Metaverse and XR [2]) and cyber-physical systems (CPSs) (e.g., autonomous driving and networked robotics [3]). For the former, semantics is a measure of the meaning of information that a human can understand; for the latter, semantics characterizes the usefulness of messages with respect to the goal of data exchange.

In CPSs, semantic-aware transmission aims to minimize the distortion and timing mismatch of the data between the edge devices and the remote destination [3]. Thus, the freshness of messages is critical, and monotonic functions of age of information (AoI) are often introduced as semantic quality metrics [4], [5]. Semantic-aware (and AoI-aware) dynamic scheduling of cyber-physical systems over limited communication resources has attracted much attention in the

communications society [1]. In [4], an optimal scheduling problem of a multi-sensor remote estimation system was considered to minimize the overall long-term estimation mean-square error (MSE). Importantly, the instantaneous MSE of each sensor is a function of the corresponding AoI. The scheduling problem was formulated as a Markov decision process (MDP) that can be solved by conventional value and policy iteration methods. In [6], an optimal AoI minimization scheduling problem was considered under an average transmission power constraint and was formulated into a constrained MDP. Note that the conventional MDP algorithms can only solve very small-scale problems due to the high computational complexity introduced by the curse of dimensionality. Heuristic scheduling policies without optimality guarantees were proposed in [7] and [8].

The recent development of deep reinforcement learning (DRL) allows us to solve large MDPs effectively with deep neural networks (NNs) for function approximations. In [9], deep Q-network (DQN) was adopted to solve optimal scheduling problems of multi-sensor-multi-channel remote estimation systems. In particular, those solutions can achieve better performance (in terms of MSE) than heuristic methods. Advanced DRL algorithms with an actor-critic framework (consisting of an actor NN and a critic NN), such as soft actor-critic (SAC) and deep deterministic policy gradient (DDPG), were introduced to deal with large-scale semantic-aware scheduling problems that cannot be handled by DQN [10], [11].

A major drawback of applying existing DRL methods in large-scale scheduling problems is that training is usually very time-consuming and can often get stuck in local minima. This motivates us to have a more profound understanding of the structural properties of optimal scheduling policies (such as the monotonicity of the critic NN), which can then be leveraged to guide DRL algorithms towards faster and more effective convergence.

In this letter, we focus on **developing advanced actor-critic DRL algorithms to optimize semantic-aware scheduling policies**. The contributions include: First, we mathematically prove that a well-trained critic NN of an optimal scheduling policy should be a **monotonic function** in terms of the AoI states and channel states of the system. Such a property has never been established in the open literature. Second, by leveraging this theoretical property, we propose two monotonic critic NN training methods: **network architecture-enabled monotonicity** and **regularization-based monotonicity**. The proposed training methods can be directly applied to the baseline DDPG and its variants. Last, our numerical results show that the proposed **monotonicity-enforced DDPG** algorithms can save training time and improve training performance significantly compared to the baseline DDPG.

Manuscript received 21 September 2023; accepted 22 October 2023. Date of publication 25 October 2023; date of current version 12 December 2023. The work of Wanchun Liu was supported by the Australian Research Council's Discovery Early Career Researcher Award (DECRA) Project DE230100016. The associate editor coordinating the review of this letter and approving it for publication was V.-D. Nguyen. (Corresponding author: Wanchun Liu.)

Jiazheng Chen, Wanchun Liu, Yonghui Li, and Branka Vucetic are with the School of Electrical and Computer Engineering, The University of Sydney, Sydney, NSW 2006, Australia (e-mail: jiazheng.chen@sydney.edu.au; wanchun.liu@sydney.edu.au; yonghui.li@sydney.edu.au; branka.vucetic@sydney.edu.au).

Daniel E. Quevedo is with the School of Electrical Engineering and Robotics, Queensland University of Technology (QUT), Brisbane City, QLD 4000, Australia (e-mail: dquevedo@ieee.org).

Digital Object Identifier 10.1109/LCOMM.2023.3327412

1558-2558 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on March 06, 2026 at 01:31:15 UTC from IEEE Xplore. Restrictions apply.

II. SYSTEM MODEL

We consider semantic communication in a CPS with N edge devices (e.g., sensors and robots) and a remote destination (e.g., a remote estimator or controller) connected by a wireless network with $M < N$ channels (e.g., sub-carriers).

A. Wireless Communication Model

At each time step t , the transmission scheduling action for device n is denoted by

$$a_{n,t} = \begin{cases} 0 & \text{if device } n \text{ is not scheduled,} \\ m & \text{if device } n \text{ is scheduled to channel } m. \end{cases} \quad (1)$$

We assume that each device can be scheduled to at most one channel and that each channel is assigned to one device, i.e.,

$$\sum_{m=1}^M \mathbb{1}(a_{n,t} = m) \leq 1, \quad \sum_{n=1}^N \mathbb{1}(a_{n,t} = m) = 1, \quad (2)$$

where $\mathbb{1}(\cdot)$ is the indicator function.

We consider independent and identically distributed (i.i.d.) block fading channels, where the channel quality is fixed during each packet transmission and varies packet by packet independently. The channel state of the system at time t is denoted by a $N \times M$ matrix $\mathbf{H}_t$, where the n th row and m th column element $h_{n,m,t} \in \mathcal{H} \triangleq \{1, 2, \dots, \hat{h}\}$ represents the channel state between device n and the destination at channel m quantized into $\hat{h}$ levels. The distribution of the channel state $h_{n,m,t}$ is given as

$$\Pr(h_{n,m,t} = j) = q_j^{(n,m)}, \forall t, \quad (3)$$

where $\sum_{j=1}^{\hat{h}} q_j^{(n,m)} = 1, \forall n, m$. A higher channel state leads to a large packet drop probability. Let $p_{n,m,t}$ denote the packet drop probability at the channel state $h_{n,m,t}$. The instantaneous full channel state $\mathbf{H}_t$ is available at the remote estimator based on standard channel estimation methods.

B. Semantic Communication Performance

First, we define $\tau_{n,t} \in \{1, 2, \dots\}$ as the AoI of device n at time t , i.e., the time elapsed since its latest device packet was successfully received [5], [6], [7]:

$$\tau_{n,t+1} = \begin{cases} 1, & \text{at time } t, \text{ device } n\text{'s packet is received,} \\ \tau_{n,t} + 1, & \text{otherwise.} \end{cases} \quad (4)$$

Next, we define a *positive non-decreasing cost* (or penalty) function $g_n(\tau_{n,t}), \forall n \in \{1, \dots, N\}$ as a semantic measurement of device n 's information. A higher AoI state corresponds to an increased cost function, indicating poorer performance in semantic communications. Semantic communication systems with varying functionalities possess distinct $g_n(\cdot)$ functions. The scheduling algorithms proposed in the rest of this letter are general and applicable to any positive and non-decreasing cost function.

An example below shows how to determine $g_n(\cdot)$ in a specific semantic communication system.

Example 1 (A remote state estimation system): Such a system consists of N sensors, each measuring a physical process, and a remote estimator for signal reconstruction. Each physical process dynamics and its measurement are modeled

as a discrete-time linear time-invariant (LTI) system:

$$\mathbf{e}_{n,t+1} = \mathbf{A}_n \mathbf{e}_{n,t} + \mathbf{w}_{n,t}, \quad \mathbf{z}_{n,t} = \mathbf{C}_n \mathbf{z}_{n,t} + \mathbf{v}_{n,t}, \quad (5)$$

where $\mathbf{e}_{n,t} \in \mathbb{R}^{l_n}$ is process n 's state at time t , and $\mathbf{z}_{n,t} \in \mathbb{R}^{c_n}$ is the state measurement of sensor n ; $\mathbf{A}_n \in \mathbb{R}^{l_n \times l_n}$ and $\mathbf{C}_n \in \mathbb{R}^{c_n \times l_n}$ are the system matrix and the measurement matrix, respectively; $\mathbf{w}_{n,t} \in \mathbb{R}^{l_n}$ and $\mathbf{v}_{n,t} \in \mathbb{R}^{c_n}$ are the process disturbance and the measurement noise modeled as i.i.d zero-mean Gaussian random vectors $\mathcal{N}(\mathbf{0}, \mathbf{W}_n)$ and $\mathcal{N}(\mathbf{0}, \mathbf{V}_n)$, respectively. Due to the noisy measurement in (5), sensor n applies a local Kalman filter (KF) to pre-process the raw measurement before sending it to the remote estimator. Due to the dynamic scheduling and the random packet dropouts, the remote estimator cannot always receive sensor n 's measurement and thus uses a minimum mean square error estimator for state estimation, say $\hat{\mathbf{e}}_{n,t}$. Then, the estimation error covariance is given as

$$\mathbf{P}_{n,t} \triangleq \mathbb{E}[(\hat{\mathbf{e}}_{n,t} - \mathbf{e}_{n,t})(\hat{\mathbf{e}}_{n,t} - \mathbf{e}_{n,t})^\top] = f_n^{\tau_{n,t}}(\bar{\mathbf{P}}_n), \quad (6)$$

where $f_n(\mathbf{X}) = \mathbf{A}_n \mathbf{X} \mathbf{A}_n^\top + \mathbf{W}_n$ and $f_n^{\tau+1}(\cdot) = f_n(f_n^\tau(\cdot))$, where $\bar{\mathbf{P}}_n$ is a constant matrix determined by $\mathbf{A}_n$, $\mathbf{C}_n$, $\mathbf{W}_n$ and $\mathbf{V}_n$.

To characterize the remote estimation performance, we define the cost function as the estimation MSE of process n

$$g_n(\tau_{n,t}) \triangleq \text{Tr}(\mathbf{P}_{n,t}) = f_n^{\tau_{n,t}}(\bar{\mathbf{P}}_n), \quad (7)$$

which has been proved to be a positive non-decreasing function of $\tau_{n,t}$, see [5] for further details.

III. SEMANTIC-AWARE TRANSMISSION SCHEDULING

We aim to find a dynamic scheduling policy $\pi(\cdot)$ that minimizes the expected total discounted cost function of all N devices over the infinite time horizon.

Problem 1:

$$\min_{\pi} \lim_{T \rightarrow \infty} \mathbb{E}^{\pi} \left[\sum_{t=1}^T \sum_{n=1}^N \gamma^t g_n(\tau_{n,t}) \right], \quad (8)$$

where $\gamma \in (0, 1)$ is a discount factor.

A. Markov Decision Process Formulation

Problem 1 is a sequential decision-making problem with the Markovian property and hence can be cast as an MDP as below.

1) The state of the MDP consists of both the AoI and channel states as $\mathbf{s}_t \triangleq (\boldsymbol{\tau}_t, \mathbf{H}_t) \in \mathbb{N}^N \times \mathcal{H}^{M \times N}$, where $\boldsymbol{\tau}_t = (\tau_{1,t}, \tau_{2,t}, \dots, \tau_{N,t}) \in \mathbb{N}^N$ is the AoI state vector.

2) The overall schedule action of the N devices is defined as $\mathbf{a}_t = (a_{1,t}, a_{2,t}, \dots, a_{N,t}) \in \{0, 1, 2, \dots, M\}^N$ under the constraint (2). The policy $\pi(\cdot)$ maps a state to an action as $\mathbf{a}_t = \pi(\mathbf{s}_t)$.

3) The immediate reward (i.e., the negative sum cost of all edge devices) at time t is defined as $-\sum_{n=1}^N g_n(\tau_{n,t})$.

4) The transition probability $\Pr(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ is the probability of the next state $\mathbf{s}_{t+1}$ given the current state $\mathbf{s}_t$ and the action $\mathbf{a}_t$. Since the optimal policy of an infinite-horizon MDP is stationary [12, Chapter 6], the state transition is independent of the time index. Thus, we drop the subscript t and use $\mathbf{s} \triangleq (\boldsymbol{\tau}, \mathbf{H})$ and $\mathbf{s}^+ \triangleq (\boldsymbol{\tau}^+, \mathbf{H}^+)$ to represent the current and

the next states, respectively. Due to the i.i.d. fading channels and the AoI state transition, we have

$$\Pr(\mathbf{s}^+|\mathbf{s}, \mathbf{a}) = \Pr(\boldsymbol{\tau}^+|\boldsymbol{\tau}, \mathbf{H}, \mathbf{a}) \Pr(\mathbf{H}^+), \quad (9)$$

where $\Pr(\boldsymbol{\tau}^+|\boldsymbol{\tau}, \mathbf{H}, \mathbf{a}) = \prod_{n=1}^N \Pr(\tau_n^+|\tau_n, \mathbf{H}, a_n)$ and

$$\Pr(\tau_n^+|\tau_n, \mathbf{H}, a_n) = \begin{cases} 1 - p_{n,m}, & \text{if } \tau_n^+ = 1, a_n = m, \\ p_{n,m}, & \text{if } \tau_n^+ = \tau_n + 1, a_n = m, \\ 1, & \text{if } \tau_n^+ = \tau_n + 1, a_n = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

B. Value Functions of the Optimal Policy

We define the V function (also called “state-value function”), $V(\mathbf{s}_t) : \mathcal{S} \rightarrow \mathbb{R}$ and the Q function (the “action-value function”), $Q(\mathbf{s}_t, \mathbf{a}_t) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ of the optimal policy $\pi^*(\cdot)$. Given the current state $\mathbf{s}_t$, the V function is the maximum expected discounted sum of the future reward achieved by the optimal policy, i.e., $V(\mathbf{s}_t) = \mathbb{E}[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(\mathbf{s}_{t'}) | \mathbf{a}_{t'} = \pi^*(\mathbf{s}_{t'}), \forall t' \geq t]$. Given the current state-action pair, $(\mathbf{s}_t, \mathbf{a}_t)$, the Q function is the maximum expected discounted sum of the future reward under the optimal policy:

$$Q(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(\mathbf{s}_{t'}) \middle| \mathbf{a}_t, \mathbf{a}_{t'} = \pi^*(\mathbf{s}_{t'}), \forall t' > t \right]. \quad (11)$$

Thus, the V function measures the long-term performance given the current state, while the Q function can be used for comparing the long-term performance affected by different current actions.

The relation between the two value functions is given by [13]

$$Q(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t) + \gamma \sum_{\mathbf{s}_{t+1}} \Pr(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) V(\mathbf{s}_{t+1}). \quad (12)$$

In particular, the Q function satisfies the Bellman equation:

$$Q(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t) + \mathbb{E}_{\mathbf{s}_{t+1}} \left[\gamma \max_{\mathbf{a}_{t+1}} Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) \right]. \quad (13)$$

C. DDPG for Scheduling Policy Optimization

A DDPG agent has two neural networks (NNs): an actor NN with the parameter set α and a critic network with the parameter set β . In particular, the actor NN aims to approximate the optimal policy $\pi^*(\mathbf{s})$ by $\pi(\mathbf{s}; \alpha)$ after well-training, while the critic NN approximates $Q(\mathbf{s}_t, \mathbf{a}_t)$ by $Q(\mathbf{s}_t, \mathbf{a}_t; \beta)$. The overall training process for the two NNs works iteratively based on sampled data named *transitions*. Each transition $\mathcal{T}_i \triangleq (\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}_{i+1})$, consists of the state, action, reward, and next state. To train the actor NN, the critic NN’s Q values are utilized for characterizing the performance of the actor NN’s policy over the sampled data; to train the critic NN, the transitions generated by the actor NN’s policy are utilized to evaluate the accuracy of Q function approximation. In particular, the approximation error given $\mathcal{T}_i$ is defined as the temporal difference (TD) error

$$\text{TD}_i^2 \triangleq (y_i - Q(\mathbf{s}_i, \mathbf{a}_i; \beta))^2, \quad (14)$$

where $y_i = r_i + \gamma \max_{\mathbf{a}_{i+1}} Q(\mathbf{s}_{i+1}, \mathbf{a}_{i+1}; \beta^-)$ is the estimation of the Q function at next step and β^- is the critic

NN parameters from the previous iteration. In other words, the training of the critic NN is to minimize the TD errors over the transitions, making the approximated Q function satisfy the Bellman equation (13) [13].

Remark 1: Our work focuses on the effective training of the critic NN by utilizing the partial monotonicity of the Q function discussed in the section below, and the training method for the actor NN is identical to the baseline DDPG. Furthermore, the proposed training methods can be applied to solve other DRL problems that involve monotonic critic NNs and can seamlessly integrate with actor-critic DRL algorithms like Twin Delayed DDPG (TD3) and Proximal Policy Optimization (PPO).

IV. PARTIAL MONOTONICITY OF Q FUNCTION

Before proceeding further, we need the following technical lemma related to the V function monotonicity.

Lemma 1 (Monotonicity of V function w.r.t. AoI States [10]): Consider states $\mathbf{s} = (\boldsymbol{\tau}, \mathbf{H})$ and $\mathbf{s}'_{\text{AoI}} = (\boldsymbol{\tau}'_{(n)}, \mathbf{H})$, where $\boldsymbol{\tau}'_{(n)} = (\tau_1, \dots, \tau'_n, \dots, \tau_N)$ and $\tau'_n \geq \tau_n$. For any $n \in \{1, \dots, N\}$, the following holds

$$V(\mathbf{s}'_{\text{AoI}}) \leq V(\mathbf{s}). \quad (15)$$

Now we establish the partial monotonicity of the Q function in terms of the AoI and channel states as below.

Theorem 1 (Monotonicity of Q Function w.r.t. AoI States): Consider action $\mathbf{a} = (a_1, \dots, a_N)$, and states $\mathbf{s} = (\boldsymbol{\tau}, \mathbf{H})$ and $\mathbf{s}'_{\text{AoI}} = (\boldsymbol{\tau}'_{(n)}, \mathbf{H})$, where $\boldsymbol{\tau}'_{(n)} = (\tau_1, \dots, \tau'_n, \dots, \tau_N)$, $\tau'_n \geq \tau_n$. The following holds

$$Q(\mathbf{s}'_{\text{AoI}}, \mathbf{a}) \leq Q(\mathbf{s}, \mathbf{a}). \quad (16)$$

Proof: The proof is based on the relation between two value functions in (12), the state transition rule in (10), and Lemma 1. See [14] for details due to space limitations. ■

Theorem 2 (Monotonicity of Q Function w.r.t. Channel States): Consider action $\mathbf{a} = (a_1, \dots, a_N)$, and states $\mathbf{s} = (\boldsymbol{\tau}, \mathbf{H})$ and $\mathbf{s}'_{\text{Ch}} = (\boldsymbol{\tau}, \mathbf{H}'_{n,m})$ where $\mathbf{H}'_{n,m}$ is identical to $\mathbf{H}$ except the device- n -channel- m state with $h'_{n,m} > h_{n,m}$.

(i) If $a_n = m$, then the following holds

$$Q(\mathbf{s}'_{\text{Ch}}, \mathbf{a}) \leq Q(\mathbf{s}, \mathbf{a}). \quad (17)$$

(ii) If $a_n \neq m$, then the equality in (17) holds, i.e., we have

$$Q(\mathbf{s}'_{\text{Ch}}, \mathbf{a}) = Q(\mathbf{s}, \mathbf{a}). \quad (18)$$

Proof: See [14] for details due to space limitations. ■

V. PARTIALLY MONOTONIC CRITIC NEURAL NETWORK

Building on the partial monotonicity of the Q function, we aim to develop novel training methods guaranteeing the monotonicity of the critic NN that can help solve Problem 1 more effectively compared to conventional learning methods. We consider two partially monotonic critic NN training methods: (i) network architecture-enabled monotonicity and (ii) regularization-based monotonicity in the sequel.

A. Network Architecture-Enabled Monotonicity

We propose a three-layer critic NN architecture (a shallow NN) that directly guarantees the partial monotonicity of the Q function, as shown in Fig. 1(a). The critic NN integrates the outputs of two NN processing the state and the action inputs separately. In particular, the state-processing part

(on the top of Fig. 1(a)) has to be a monotonic decreasing function. We adopt the monotonic NN in [15], which guarantees that the output is a monotonic decreasing function of the input with the following conditions: 1) the activation functions are non-decreasing (e.g., sigmoid or ReLU), and 2) the weights satisfy

$$\beta_{i,\bar{j}}^{(1,2)} \beta_{\bar{j}}^{(2,3)} \leq 0, \forall i \in \{1, N(M+1)\}, \bar{j} \in \{1, \dots, U\}. \quad (19)$$

In (19), $\beta_{i,\bar{j}}^{(1,2)}$ denotes the weight for the connection between $\bar{i}$ -th input node and the $\bar{j}$ -th hidden node, $\beta_{\bar{j}}^{(2,3)}$ denotes the weight for the connection between the $\bar{j}$ -th hidden node and the output node in the critic NN, and U is the size of the hidden layer. There is no specific requirement on the parameters of the action-processing part (at the bottom of Fig. 1(a)).

First, we adopt the sigmoid activation function with positive output. Then, to satisfy the constraint (19) and to ensure negative Q value outputs (see the definition in (11)) during training, we restrict the weights $\beta_{i,\bar{j}}^{(1,2)}$ and $\beta_{\bar{j}}^{(2,3)}$ to be non-negative and non-positive, respectively, i.e., $\beta_{i,\bar{j}}^{(1,2)} \leftarrow \max(0, \beta_{i,\bar{j}}^{(1,2)})$ and $\beta_{\bar{j}}^{(2,3)} \leftarrow \min(0, \beta_{\bar{j}}^{(2,3)})$.

The advantage of the presented approach is that the monotonicity is strictly and automatically ensured by the NN architecture during the training at each time step. A potential drawback is that the “shallow” monotonic critic NN has a single hidden layer, limiting its capability for approximating complex functions, which are required when solving scheduling problems with large state and action spaces. Note that although there are some deep monotonic NN, such as deep lattice networks [16], these NNs have very complex architectures and are computationally expensive for large inputs. Therefore, we propose the following approach for training a “deep” monotonic critic NN, which is a multi-layer fully connected NN without a specific NN architecture requirement.

B. Regularization-Based Monotonicity

We introduce penalty terms in the loss function of the critic NN training to enforce a monotonically decreasing relationship between the state input and the output. In particular, we propose two types of penalty for monotonicity regularization.

1) *Positive Derivative Penalty (Type I)*: Let $s_{i,j}$ denote the j -th state of the length- $N(M+1)$ state vector s_i , which can be an AoI state (i.e., $j \leq N$) or a channel state (i.e., $j > N$). The partial derivative of the Q function $Q(s_i, \mathbf{a}_i; \beta)$ at the state input $s_{i,j}$ is given as

$$QD_{i,j} = \nabla_{s_{i,j}} Q(s_i, \mathbf{a}_i; \beta) \quad \forall j \in \{1, \dots, N(M+1)\}. \quad (20)$$

To enforce the critic NN approximating a monotonically decreasing function at $s_{i,j}$, the corresponding positive derivative penalty is $\max(0, QD_{i,j})$.

The computation of the derivative (20) requires backward propagation, which has a higher computational complexity than forward propagation, i.e., evaluating a Q value by feeding the critic NN with an input. Thus, we propose a forward-propagation-only penalty scheme below.

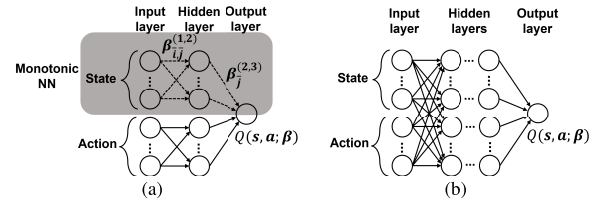


Fig. 1. Partially monotonic critic NNs. (a) A shallow critic NN with network architecture-enabled monotonicity, where the NN has only one hidden layer. (b) A deep critic NN with regularization-based monotonicity.

2) *Positive Increment Penalty (Type II)*: We introduce the Q value increment from states s_i to $s'_{i,j}$ as

$$Ql_{i,j} = Q(s'_{i,j}, \mathbf{a}_i; \beta) - Q(s_i, \mathbf{a}_i; \beta), \quad (21)$$

where $s'_{i,j}$ is identical to s_i except for the j -th state with one-step increment, $s'_{i,j} = s_{i,j} + 1$. To enforce the critic NN approximating a monotonically decreasing function from s_i to $s'_{i,j}$, we define the positive increment penalty as $\max(0, Ql_{i,j})$.

Potentially, given the vector state s_i and based on the penalty defined above related to $s_{i,j}$, one can calculate penalty terms for all $j \in \{1, \dots, N(M+1)\}$, leading to high computational cost when N and M are large. Therefore, we propose a novel scheme to compute the penalty effectively.

Stochastic Penalty Sampling Scheme: First, given the state action pair $(s_i, \mathbf{a}_i)$, we define an effective set $\mathcal{J}_i \subset \{1, \dots, N(M+1)\}$ of the individual states for penalty calculation. From (20) and (21), a penalty term is effective if $QD_{i,j}$ or $Ql_{i,j}$ is positive and the Q value changes dramatically. From (18) of Theorem 2, channel $h_{n,m}$'s quality improvement does not change the Q value at all, if device n is not scheduled at channel m . Thus, we can eliminate those cases when calculating the penalty. We define an indicator on whether a channel state $s_{i,j}$ with $j > N$ should be considered for penalty evaluation or not, i.e.,

$$I(s_{i,j}) = \begin{cases} j & \mathbf{a}_i \text{ utilizes } s_{i,j} \text{'s corresponding channel,} \\ \emptyset & \text{otherwise.} \end{cases} \quad (22)$$

Then, the effective set for penalty calculation is

$$\mathcal{J}_i = \{1, \dots, N, I(s_{i,N+1}), \dots, I(s_{i,N(M+1)})\}. \quad (23)$$

Second, we randomly generate a uniformly down-sampled set $\bar{\mathcal{J}}_i$ with size $K \ll N(M+1)$ based on $\mathcal{J}_i$. From (20) and (21), the loss function of the transition $\mathcal{T}_i$ regularized by the penalty terms determined by $\bar{\mathcal{J}}_i$ is

$$L_i(\beta) = \begin{cases} TD_i^2 + \sum \max(0, QD_{i,j}) & \text{type I regularized,} \\ TD_i^2 + \sum_{j \in \bar{\mathcal{J}}_i} \max(0, Ql_{i,j}) & \text{type II regularized.} \end{cases} \quad (24)$$

A well-trained critic NN β should minimize both the TD error and the penalty term in the loss function above.

VI. NUMERICAL EXPERIMENTS

In this section, we evaluate and compare the performance of the three monotonicity-enforced training methods for DDPG proposed in the previous section, namely monotonic architecture-based DDPG (MA-DDPG), type I monotonicity-regularized DDPG (MRI-DDPG), and type II

TABLE I
PERFORMANCE COMPARISON OF THE DIFFERENT DDPG ALGORITHMS

System Scale (N, M)	Algorithms	Training time/episode (second)	Experiment 1		Experiment 2		Experiment 3		Experiment 4		Experiment 5	
			NEC	MSE	NEC	MSE	NEC	MSE	NEC	MSE	NEC	MSE
(6, 3)	DDPG	3.10	—	—	—	—	107	133.7559	—	—	—	—
	MA-DDPG	3.17	58	50.4761	80	80.3567	30	56.3479	56	66.5171	83	74.1661
	MRI-DDPG	4.69	74	57.3178	—	—	82	61.4528	—	—	—	—
	MRII-DDPG	4.01	66	57.4541	—	—	53	61.4805	—	—	—	—
(14, 7)	DDPG	6.16	189	160.6451	162	136.5146	—	—	—	—	132	127.8284
	MRI-DDPG	8.15	148	122.1874	105	114.3175	204	140.6513	176	207.1918	97	110.1273
	MRII-DDPG	7.20	87	112.3492	60	108.8440	160	131.0342	113	187.2761	68	103.4623
(20, 10)	DDPG	8.32	193	210.6116	188	199.5522	213	175.2343	—	—	—	—
	MRI-DDPG	9.83	158	172.7708	163	181.6881	182	144.6849	127	180.3464	273	216.3028
	MRII-DDPG	9.37	116	155.7731	109	170.6473	131	134.4982	103	167.6880	201	181.9388

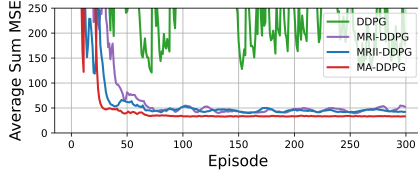


Fig. 2. Average sum MSE during training with $N = 6, M = 3$. The critic NN has only one hidden layer – a shallow NN setup.

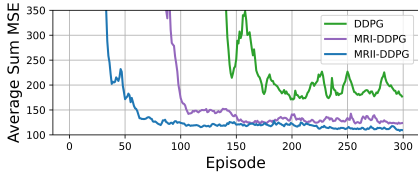


Fig. 3. Average sum MSE during training with $N = 14, M = 7$. The critic NN has three hidden layers – a deep NN setup.

monotonicity-regularized DDPG (MRII-DDPG). The experiments are built on the remote estimation system described in Example 1. Due to the space limitation, the parameter setups are given in [14].

Figs. 2 and 3 show the average sum MSE of all processes during the training achieved by different DDPG algorithms with $N = 6, M = 3$ and $N = 14, M = 7$, respectively. Fig. 2 illustrates that the MA-DDPG reduces the average sum MSE by 15% when compared to the MRI-DDPG and MRII-DDPG, while the baseline DDPG cannot converge with the shallow critic NN. Thus, for a small-scale system, MA-DDPG is the most effective algorithm. Fig. 3 shows that MRI-DDPG and MRII-DDPG save about 15% and 50% training episodes for convergence, respectively, and also reduce the average sum MSE by 30% when compared to DDPG. MRII-DDPG performs better than MRI-DDPG, since the state space is discrete, and MRII-DDPG directly regularizes the Q function monotonicity at the discrete state space. Note that MA-DDPG cannot converge due to its shallow NN architecture.

In Table I, we evaluate the training time, the number of episodes for convergence (NEC), and the performance of different DDPG algorithms (based on 20000-step simulations) over different system scales and parameters. We see that for the 20-sensor-10-channel systems, MRII-DDPG can reduce the average MSE by 27% when compared to DDPG (at the cost of the increased training time per episode by 13%), whilst saving more than 30% time for convergence (i.e., training time/episode $\times$ NEC), despite the extended training duration of each episode. In particular, MRII-DDPG solves the scheduling problem in Experiments 4 and 5 effectively, while the baseline DDPG fails to converge.

VII. CONCLUSION

In this work, building on the monotonicity of the Q function of the optimal semantic-aware scheduling policy, we have developed monotonicity-driven DRL algorithms to solve the scheduling problems effectively and reduce the MSE by about 30% compared to the conventional DRL algorithm. For future work, we will investigate semantic-aware resource allocation problems over practical time-correlated fading channels.

REFERENCES

- [1] D. Gündüz et al., “Beyond transmitting bits: Context, semantics, and task-oriented communications,” *IEEE J. Sel. Areas Commun.*, vol. 41, no. 1, pp. 5–41, Jan. 2023.
- [2] J. Chen et al., “Age of incorrect information in semantic communications for NOMA aided XR applications,” *IEEE J. Sel. Topics Signal Process.*, early access, Jun. 5, 2023, doi: [10.1109/JSTSP.2023.3282836](https://doi.org/10.1109/JSTSP.2023.3282836).
- [3] M. Kountouris and N. Pappas, “Semantics-empowered communication for networked intelligent systems,” *IEEE Commun. Mag.*, vol. 59, no. 6, pp. 96–102, Jun. 2021.
- [4] S. Wu et al., “Optimal scheduling of multiple sensors over shared channels with packet transmission constraint,” *Automatica*, vol. 96, pp. 22–31, Oct. 2018.
- [5] W. Liu et al., “Remote state estimation with smart sensors over Markov fading channels,” *IEEE Trans. Autom. Control*, vol. 67, no. 6, pp. 2743–2757, Jun. 2022.
- [6] H. Tang et al., “Minimizing age of information with power constraints: Multi-user opportunistic scheduling in multi-state time-varying channels,” *IEEE J. Sel. Areas Commun.*, vol. 38, no. 5, pp. 854–868, May 2020.
- [7] Z. Qian et al., “Minimizing age of information in multi-channel time-sensitive information update systems,” in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jul. 2020, pp. 446–455.
- [8] J. Tong et al., “Age-of-information oriented scheduling for multichannel IoT systems with correlated sources,” *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9775–9790, Nov. 2022.
- [9] L. Yang et al., “Optimal sensor scheduling for remote state estimation with limited bandwidth: A deep reinforcement learning approach,” *Inf. Sci.*, vol. 588, pp. 279–292, Apr. 2022.
- [10] J. Chen et al., “Structure-enhanced DRL for optimal transmission scheduling,” *IEEE Trans. Wireless Commun.*, early access, May 25, 2023, doi: [10.1109/TWC.2023.3277861](https://doi.org/10.1109/TWC.2023.3277861).
- [11] M. Akbari et al., “Age of information aware VNF scheduling in industrial IoT using deep reinforcement learning,” *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2487–2500, Aug. 2021.
- [12] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.
- [13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [14] J. Chen et al., “Semantic-aware transmission scheduling: A monotonicity-driven deep reinforcement learning approach,” 2023, *arXiv:2305.13706*.
- [15] H. Daniels and M. Velikova, “Monotone and partially monotone neural networks,” *IEEE Trans. Neural Netw.*, vol. 21, no. 6, pp. 906–917, Jun. 2010.
- [16] S. You et al., “Deep lattice networks and partial monotonic functions,” in *Proc. NeurIPS*, 2017, pp. 1–9.